

AI 스마트 미러 피트니스 코칭 시스템 -Groom

PC1-PC2-PC3 분산 구조 기반 운동 분석 및 AI 코칭

목차

01. 문제 정의

02. 전체 시스템 구조

03. PC1 구성

04. PC2 구성

05. PC3 구성

06. 시연 및 결론

문제 정의

01 자세 확인 어려움

운동 중 본인의 자세가 올바른지 실시간으로 확인하기 어렵다.

02 개인 맞춤 루틴 부족

운동 경험, 목표, 제한 부위에 맞는 계획을 매번 만들기 어렵다.

03 운동 후 피드백 부족

운동 결과가 다음 루틴과 코칭으로 자연스럽게 이어지기 어렵다.

✅ 해결 방향: 카메라 기반 자세 분석 + AI 루틴 생성 + 운동 후 코칭 피드백



이걸 해결해주는
똑똑한 거 없을까?



아이디어 정리



개발 과정



PC 별 역할 분리

역할을 분리하면 책임과 장애 범위의 명확성 상승, 확장 또한 용이.

1

역할 분리

PC1·PC2·PC3가 각각 화면,
AI 생성, 분석·제어를 독립 담당

2

장애 범위 분리

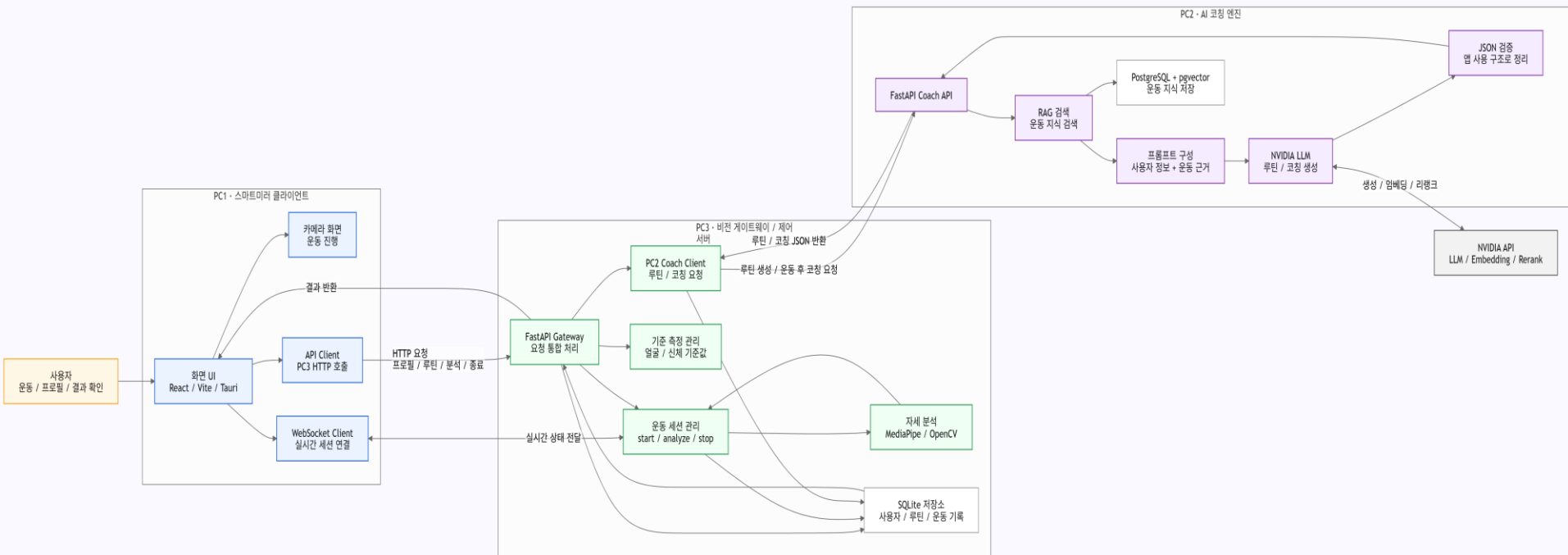
AI 서버가 지연되거나 실패해도
PC1 화면과 PC3의 세션·사용자
데이터 흐름은 분리되어 유지

3

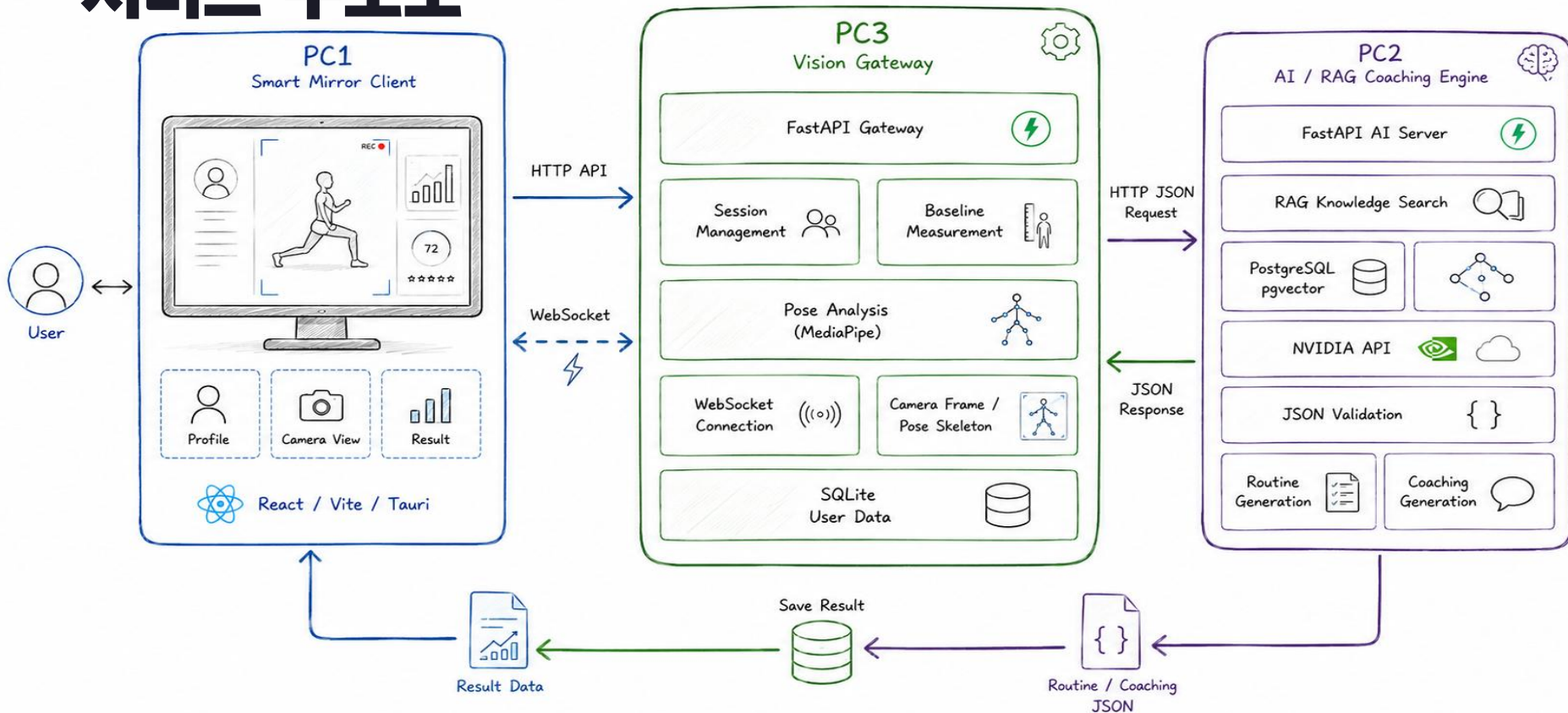
확장 가능성

운동 종류, AI 모델, 저장 구조를
PC별로 독립적으로 교체하거나
확장하기 쉬운 구조.

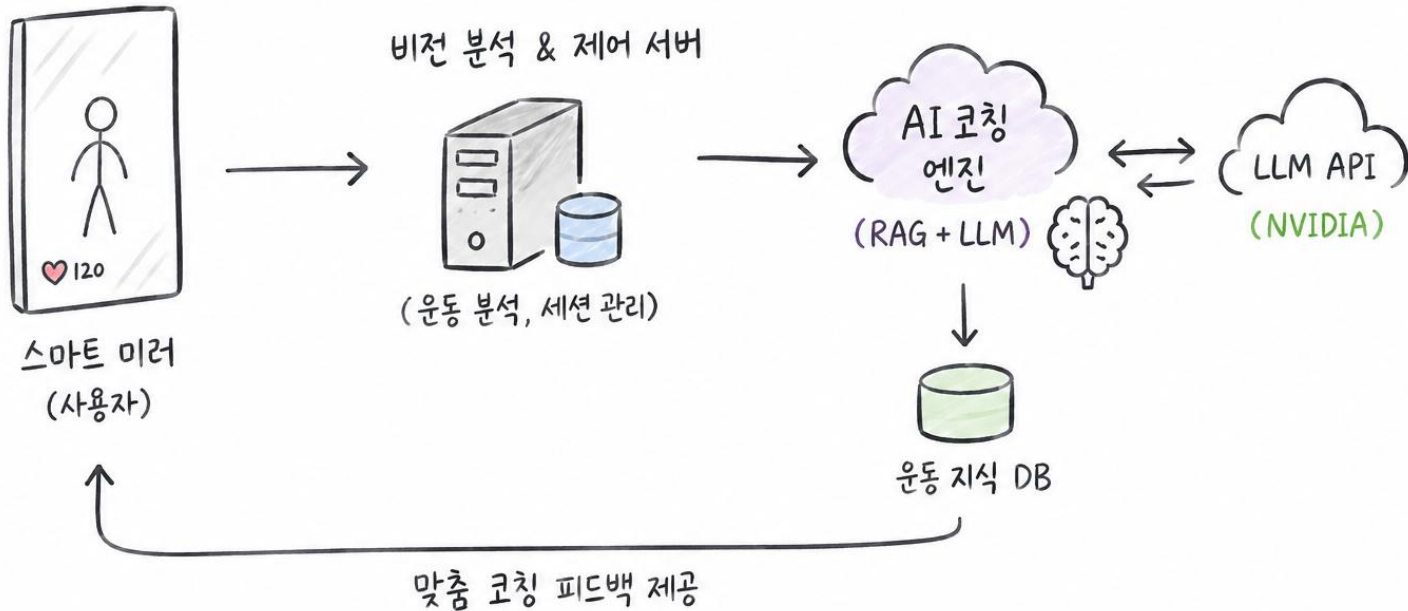
서비스 구조 머메이드 차트



서비스 구조도



AI 스마트 미러 피트니스 코칭 시스템



전체 시스템 아키텍처

PC1은 PC3만 호출하고, PC3가 데이터와 분석을 관리하며 PC2 AI 서버를 호출한다.



프로필 생성

사용자→PC1 프로필 입력 → PC3 저장 (POST /api/users/profiles)



기준 측정

PC1→PC3 baseline 캡처 → PoseAnalyzer 검증 → DB 저장 (POST /api/baselines/capture)



루틴 생성

PC3 이력 조회 → PC2 RAG+NVIDIA LLM 호출 → JSON 검증 → PC3 저장 → PC1 표시 (POST /api/routines/profile)



운동 진행

PC1→PC3 세션 시작 → PoseAnalyzer 프레임 분석 → PC1 실시간 피드백 (POST /api/sessions/start)



코칭 생성

PC1→PC3 세션 종료 → PC2 코칭 생성 → RAG+NVIDIA → PC3 저장 → PC1 코칭 표시 (POST /api/coach/generate)

PC1 · Smart Mirror UI

React · Vite · Tauri · TypeScript

PC3 · Vision Gateway

FastAPI · SQLite · MediaPipe · OpenCV

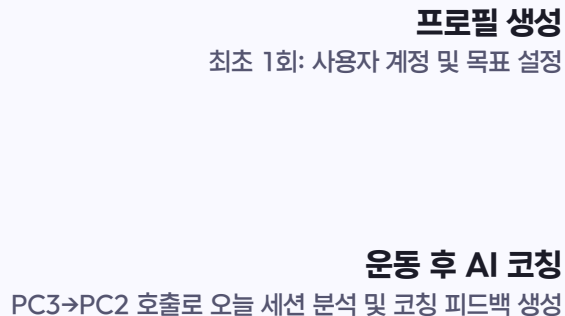
PC2 · AI/RAG Engine

pgvector · NVIDIA API · 운동 지식 DB

❗ PC1 → PC3 순서로만 흐르며, PC1이 PC2를 직접 호출하는 경우는 없습니다.

PC1 서비스 구조

사용자가 운동을 진행하는 화면 앱이며, 모든 데이터 요청은 PC3로 전달됩니다.



기준 측정

최초 & 주간 재측정: 스쿼트 자세로 체력 베이스라인 측정

루틴 확인

AI가 생성한 주간 운동 루틴 검토

운동 시작

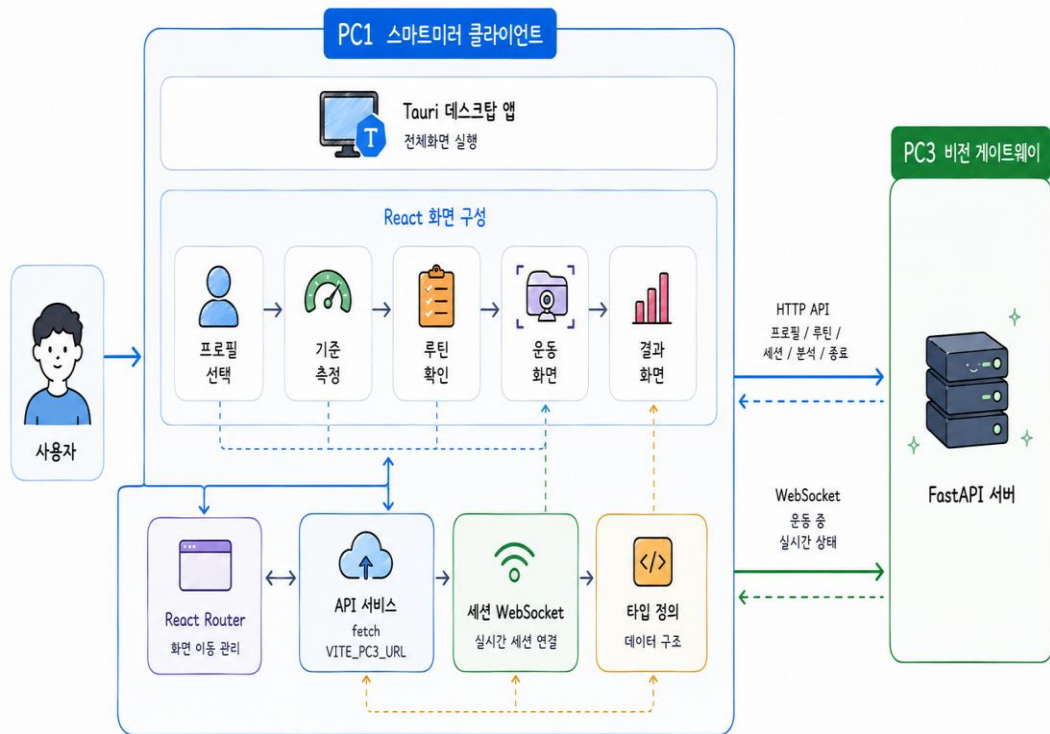
카메라로 자세 실시간 분석

🕒 1주일 반복: 루틴 확인 → 운동 시작 → 운동 후 AI 코칭을 매일 반복합니다.

🔄 주간 사이클: 1주 후 기준 측정부터 다시 시작하여 루틴을 갱신합니다.

PC1 구조

PC1 사용자 인터페이스 핵심 역할



사용자 화면 구성

프로필 선택, 기준 측정, 루틴 확인, 운동 화면, 결과 화면을 제공

PC3 API 호출

VITE_PC3_URL 기준으로 루틴 생성, 세션 시작, 운동 분석 요청을 PC3에 전달

실시간 상태 반영

운동 중 PC3 WebSocket으로 세션 상태와 분석 결과를 받아 화면에 표시

PC1 소스코드 및 기술스택

PC1 → PC3 API 호출 구조

```
// src/services/api.ts
// ① PC3 서버 주소 분리
const BASE = import.meta.env.VITE_PC3_URL
  ?? 'http://127.0.0.1:9000';

// ② 루틴 생성 요청을 PC3로 전달
export async function getRoutine(
  profile, baselineCompleted
) {
  return request('/api/routines/profile', {
    method: 'POST',
    body: JSON.stringify(body),
  });
}

// ③ 세션 시작 후 WebSocket 연결 정보 수신
export async function startSession(
  userId, goal, routineId, routineDayId
) {
  return request('/api/sessions/start', {
    method: 'POST',
    body: JSON.stringify(body),
  });
}
```

PC1 기술 스택

React

프로필 선택, 기준 측정, 운동 화면, 결과 화면을 컴포넌트 단위로 구성

Vite

개발 서버 및 빌드 도구. 빠른 화면 확인과 배포 파일 생성

Tauri

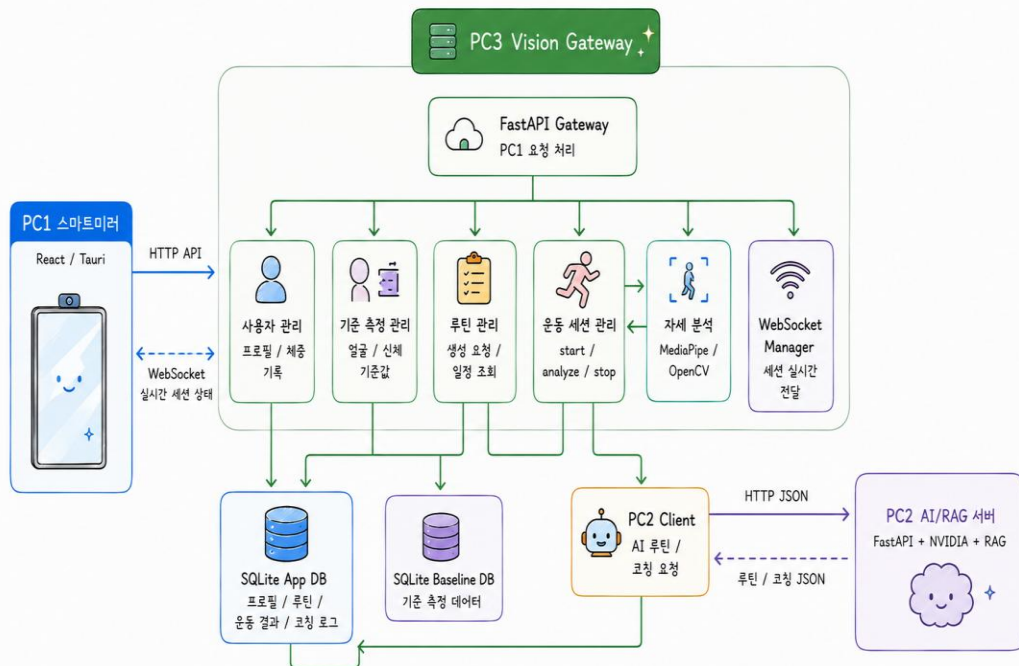
PC1 화면을 Windows 데스크톱 앱처럼 전체화면 실행 및 설치 파일 생성

TypeScript

PC3와 주고받는 응답 구조를 타입으로 관리해 필드 오류를 줄임

PC3 구조

PC3는 사용자 데이터 저장, 자세 분석과 PC2 호출을 함께 관리.



사용자 데이터 원본

프로필, baseline, 루틴, 운동 결과, 코칭 로그를 저장
Sqlite

자세 분석

MediaPipe/OpenCV 기반 프레임 분석과 baseline 검증

PC2 호출 제어

루틴 생성과 운동 후 코칭 요청을 PC3에서 통제

PC3 소스코드 기술스택

루틴 생성 요청 → PC2 호출 → PC3 저장

```
@router.post("/profile", response_model=RecommendationResponse)
// ① PC1에서 루틴 생성 요청 획득
async def generate_profile_routine(
    payload: dict = Body(...),
    baseline_service: BaselineService = Depends(get_baseline_service),
    coach_client: CoachClient = Depends(get_coach_client),
    app_store: AppStore = Depends(get_app_store),
) -> RecommendationResponse:
    request = _normalize_routine_request(payload)

    // ② 사용자 프로필과 기준 측정 데이터를 검증
    _validate_profile(request)
    _validate_saved_baseline(request.user_id, baseline_service)

    app_store.upsert_profile(_profile_for_store(request))

    // ③ 기존 운동 기록을 조회해 PC2에 함께 전달
    user_history = app_store.get_user_history_snapshot(request.user_id)

    generated = await coach_client.generate_routine(
        request,
        user_history=user_history,
    )

    // ④ PC2가 생성한 루틴을 PC3 SQLite에 저장
    stored = app_store.save_routine_plan(request, generated)
    return RecommendationResponse.model_validate(stored)
```

PC3 기술 스택

FastAPI

API 서버를 빠르게 만들 수 있음

WebSocket

운동 중 실시간 상태 전달

MediaPipe/OpenCV

카메라 프레임에서 관절 좌표를 추출하고 자세 분석

SQLite

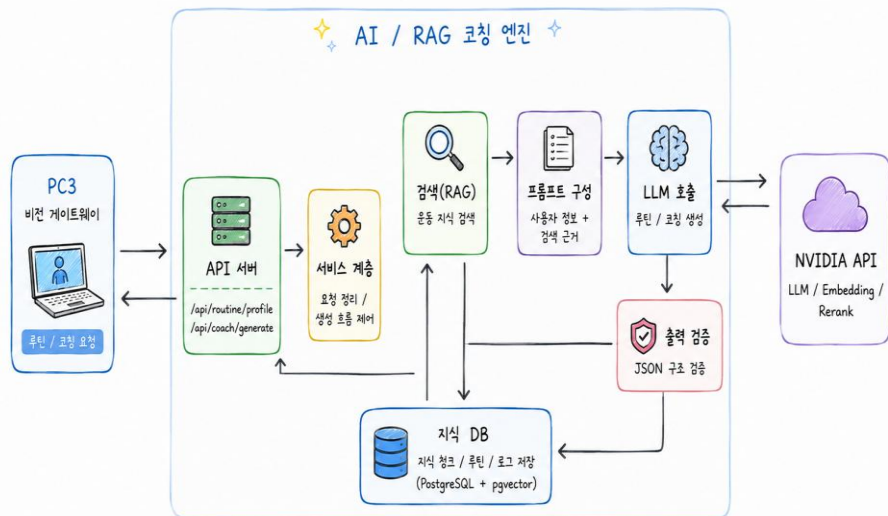
별도 DB 서버 없이 사용자 프로필, 기준 측정, 루틴, 운동 결과, 코칭 로그 저장

httpx / Pydantic

PC2 서버를 HTTP로 호출하고 JSON 요청·응답 구조 검증

PC2 구조

PC2는 운동 지식을 검색하고 NVIDIA 모델을 호출해 루틴과 코칭 JSON을 생성.



AI/RAG 생성 엔진

루틴 생성과 운동 후 코칭 생성을 담당합니다. RAG로 운동 근거를 찾고 NVIDIA LLM을 호출

운동 지식 저장·검색

운동 가이드라인과 연구 자료를 지식 청크로 저장하고, PostgreSQL/pgvector로 관련

구조화된 JSON 반환

LLM 응답을 앱에서 바로 쓸 수 있도록 루틴/코칭 JSON 구조로 검증해 PC3에 반환

PC2

PC2 생성 파이프라인

```
// ① PC3에서 전달된 사용자 정보를 정리.
def generate_profile_routine_response(payload, logger) -> dict:
    request_payload = dump_model(payload)
    history = payload.user_history or get_user_history_snapshot(payload.user_id)

// ② RAG로 운동 지식 근거를 검색.
    evidence_block, evidence = retrieve_routine_evidence(request_payload)

// ③ 검색 근거와 사용자 정보를 프롬프트로 구성
    system_prompt, user_prompt = build_profile_routine_prompt(
        request_payload,
        evidence_block,
        _build_user_history_block(history),
        _build_weekly_adjustment_block(weekly_adjustment),
    )

// ④ NVIDIA LLM을 호출해 루틴 JSON을 생성.
    llm_result = call_primary_llm(system_prompt, user_prompt)
    raw_llm = llm_result["content"]

// ⑤ 응답 구조를 검증한 뒤 PC3로 반환.
    final_response = parse_profile_routine_json(raw_llm, base_response)
    return final_response
```

PC2 기술 스택

FastAPI

PC3에서 오는 루틴/코칭 생성 요청을 API로 처리

RAG / pgvector

운동 지식 청크를 검색해 사용자 상황에 맞는 근거 제공

NVIDIA API

검색 근거와 사용자 정보를 바탕으로 루틴/코칭 JSON 생성

Pydantic / Validator

LLM 응답을 검증해 PC3가 사용할 수 있는 구조로 반환

2조 - 스마트 미래 시연 영상

한계점

실사용 안정성

카메라 각도, 조명, 사람 위치에 따라 분석 품질이 달라질 수 있음

물리적 서버 분리 환경에서 IP, WebSocket, 네트워크 지연 안정화 필요

운동 종류 제한

운동별 기준값, 오류 유형, 코칭 문장을 지속적으로 보강 필요

향후 발전계획 및 결론

1

실사용 안정화

카메라·조명·네트워크
조건별 검증과 예외 처리 강화

2

운동 종류 확대

현 5종 외 다양한 동작의 기준값과
피드백 규칙 확장

3

기록 기반 추천

RAG 규칙 강화, 임베딩 소스 추가
(대한칼리스데닉스협회(CAOS))